System Specifications

3

Application Environment

4

Application Interface Layer

1

Summary

This Chapter specifies the structure and functioning of servers for the Objects which form the interface between the Application Layer and the Application and Management.

Version 02.01.01 is a KNX Approved Standard.

Document updates

Version	Date	Modifications
AS v1.0	2001.12.19	Preparation of the Approved Standard.
AS v1.1	2009.01.05	Finalisation of the Approved Standard v1.1.
AS v1.01.01	2011.09.16	Minor editorial corrections.
01.01.02	2013.10.28	Editorial updates for the publication of KNX Specifications 2.1.
02.01.01	2019.12.18	• AN163 “Extended Interface Object addressing” integrated.
02.01.01	2020.01.03	• AN158 “KNX Data Security” integrated.
02.01.01	2021.05.19	• AN193 “Access Policies” integration started.
	2021.09.28	Preparation for inclusion in the KNX Specifications v3.0

References

[01] Chapter 3/2/3 “Powerline” v02.02.02
[02] Chapter 3/2/6 “KNX IP”
[03] Chapter 3/3/4 “Transport Layer”
[04] Chapter 3/3/7 “Application Layer”
[05] Part 3/5 “Management”
[06] Chapter 3/5/1 “Resources”
[07] Chapter 3/5/2 “Management Procedures”
[08] Chapter 3/6/3 “External Message Interface”
[09] Chapter 3/7/1 “Interworking Model”
[10] Chapter 3/7/2 “Datapoint Types”
[11] Chapter 3/7/3 “Standard Identifier Tables”
[12] Volume 6 “Profiles”
[13] Volume 7 “Application Descriptions”

Filename: 03_04_01 Application Interface Layer AS v02.01.01.docx
Version: 02.01.01
Status: Approved Standard
Savedate: 2021.09.28
Number of pages: 44

Contents

1 Overview

The Application Interface Layer shall be the layer between the Application Layer and the Application. It shall contain the communication relevant tasks of the application. It shall ease the communication task of the application by offering a communication interface that abstracts from many Application Layer details.

The KNX System allows single-processor and dual-processor device designs. A dual processor device uses additional services to communicate via a serial External Message Interface (EMI) with the external User Application running in the second processor. The EMI is specified in [06].

The following clauses define the client and server functionality and the communication interface of the internal user application located in the BAU.

The Application Interface Layer shall contain the following objects and the communication modes to them. For the specification of the communication modes, please refer to [01].

- **Group Objects**

Group Objects shall be accessible via TSAPs and ASAPs via Application Layer services on

- point-to-multipoint, connectionless (multicast) communication mode

Group Objects may also be references to Interface Objects.

- **Interface Objects**

Interface Objects shall be accessible via Application Layer services on

- point-to-point connectionless communication mode, and
- point-to-point connection-oriented communication mode, and
- point-to-domain connectionless (broadcast) communication mode.

The Interface Objects are classified as System Interface Object or Application Interface Object.

▲ System Interface Objects

This class of Interface Objects is relevant for network management and device management (see [05]). The System Interface Objects are specified in [04].

EXAMPLES the Device Object,
the Group Address Table Object,
the Group Object Association Table Object
the Application Object

▲ Application Interface Objects

These shall be Interface Objects defined in the user application. They may be defined by the internal or external user application, based on Interface Object structure rules specified in this document. Application Interface Objects may also be referenced by a Group Object reference.

The following clauses explain the data structures of each of the Application Interface Layer objects. Additionally they define by which Application Layer services these objects shall or may be accessible. Both the object client and object server functionality may be implemented by the external or the internal Application Interface Layer. It is recommended to locate the Group Objects and the Interface Objects in the internal Application Interface Layer.

2 Object models

In the KNX System, two different kinds of objects are supported for operational exchanges:

- Group Objects [1)]

  These objects shall serve for a communication model named “Shared Variable Model”.

- Interface Objects

  These objects shall serve for a communication model according the client / server model and - if they are referenced by Group Objects - also the Shared Variable Model of the Group Objects

An application can use each kind of objects at any time.

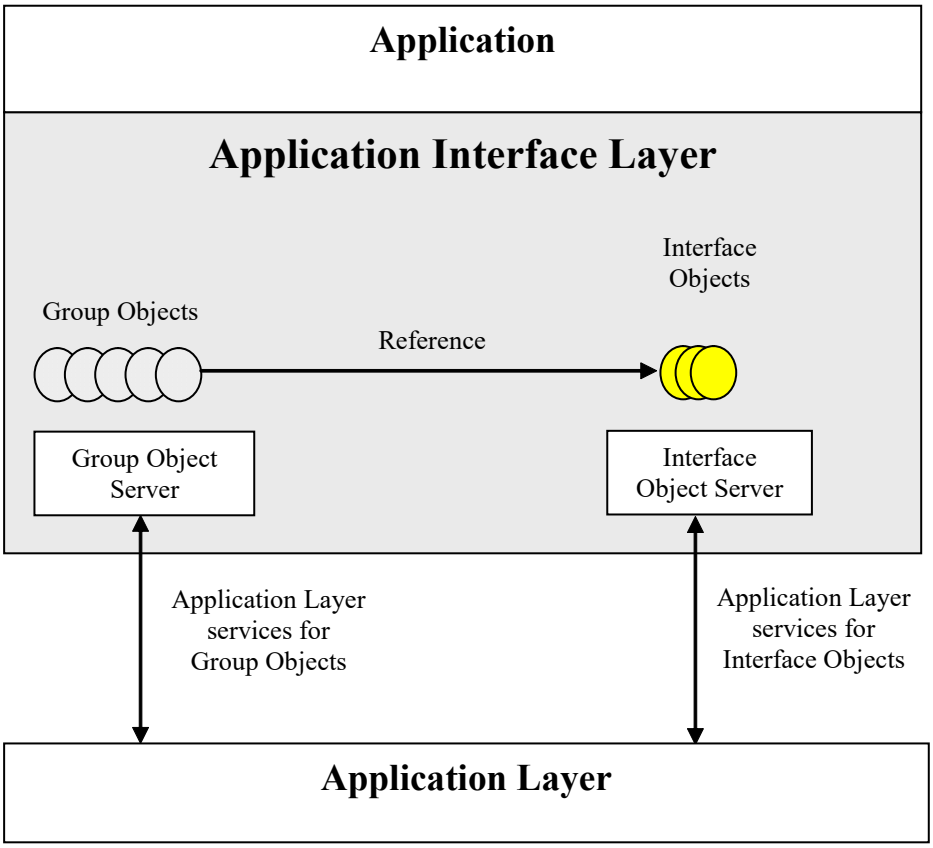


Figure 1 – AIL model

[1)] Group Objects are also accessible via the External Message Interface with a special set of services (A_Value_Read, A_Value_Write, A_Flags_Read and A_Flags_Write) that are specified in [06].

3 Group Object Server

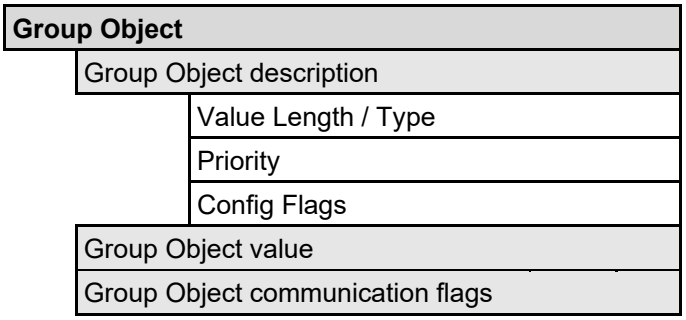
3.1 Overview

Group Objects may be distributed to a number of devices. Each device may be transmitter and receiver for Group Object values. More than one Group Object may exist in an end device and a Group Object in an end device may be assigned to one or more Group Addresses. Group Objects of an end device may belong to the same or to different groups. Each group has a network wide unique Group Address. The Group Address shall be mapped to a local group-index (TSAP) that shall be unique for the communication services of the end device by the Transport Layer. The Application Layer shall map the group-index by the Group Object Association Table to the group reference ID (ASAP) that shall access the Group Objects.

3.2 General data structure of the Group Object Table

In the sense of the previous clause a Group Object shall consist of three parts:

1. the Group Object description,
2. the Group Object value and
3. the Group Object communication flags.



The Group Object description must at least include the Group Object Type and the transmission priority.

The Config Flags shall include static information about the Group Object:

1. Read Enable
2. Write Enable
3. Transmit Enable
4. Communication Enable
5. Update Enable

The priority is urgent, normal or low.

The following Group Object Types are defined.

Group Object Type	Size of the Group Object Value
Unsigned Integer (1)	1 bit
Unsigned Integer (2)	2 bits
Unsigned Integer (3)	3 bits
Unsigned Integer (4)	4 bits
Unsigned Integer (5)	5 bits
Unsigned Integer (6)	6 bits
Unsigned Integer (7)	7 bits
Unsigned Integer (8)	1 octet
Unsigned Integer (16)	2 octets
Octet (3)	3 octets
Octet (4)	4 octets
Octet (6)	6 octets
Octet (8)	8 octets
Octet (10)	10 octets
Octet (14)	14 octets
Interface Object Reference	4 octets to 14 octets

Figure 2 - Group Object Types

Only Group Objects of the same Group Object Type shall be linked to the same Group Address and for Interface Objects references also the Interface Object Type with the same instance number must be the same.

The interpretation of the Group Object Value (data) is specified in [07].

The communication flags shall show the state of a Group Object. Following states shall be possible:

1. update
2. read_request
3. write_request
4. transmitting
5. ok, error

3.3 Group Object value transfers

3.3.1 Functionality

The application process shall trigger Group Object value transfers by "setting" or "clearing" the relevant communication flags of a Group Object. The Group Objects, or their images, shall be held in the Group Object Server. The communication flags shall play an essential role in triggering the Application Interface Layer's Group Object service to initiate the transfers. The local access to a Group Object of the Group Object server shall stimulate the Group Object server to initiate a network wide update of that Group Object. Complementary, if an update has been received, the local application shall be triggered to use the new value. There are four cases to be considered:

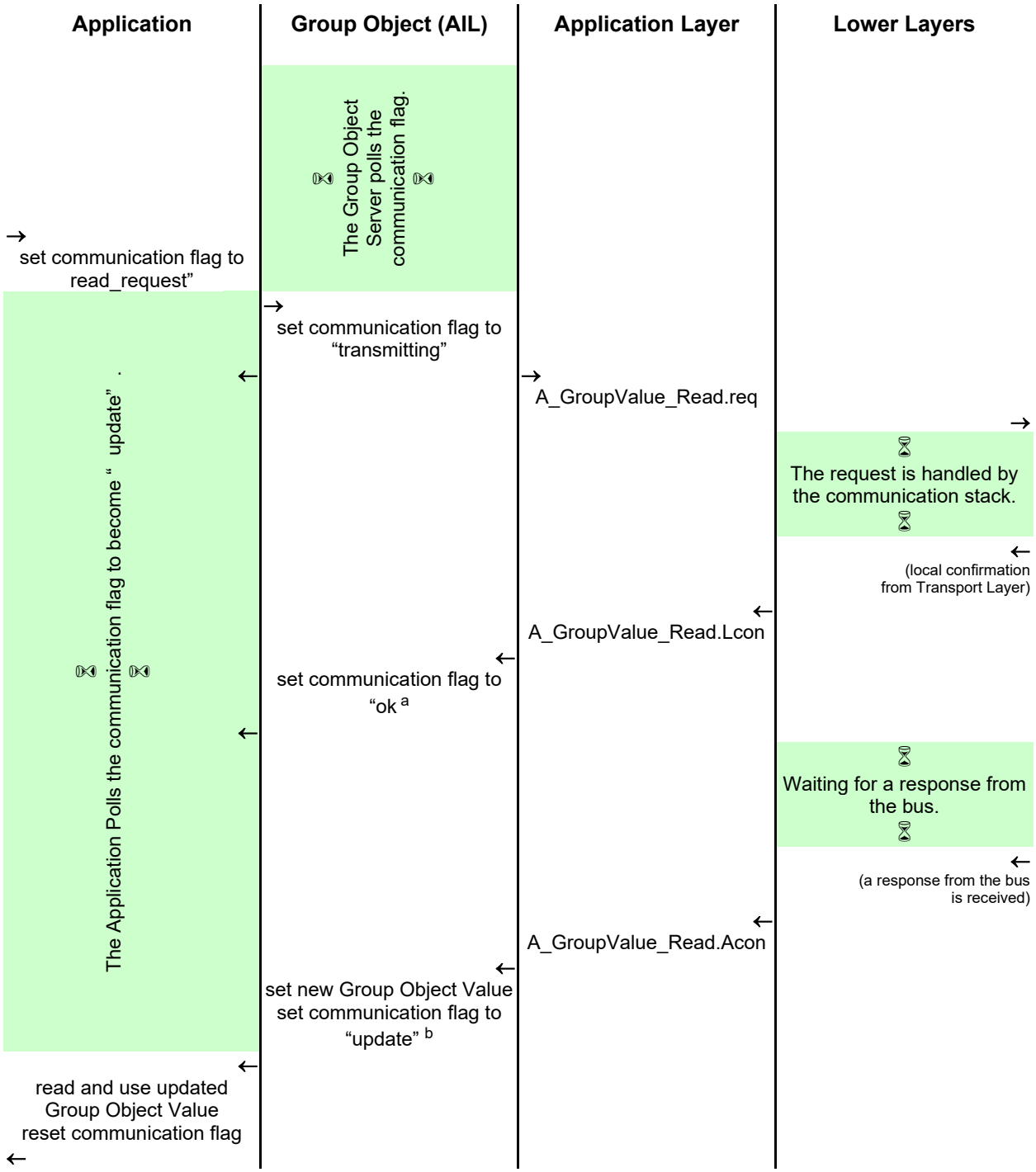
- the application wants
    1. to read the Group Object's value
    2. to write the Group Object's value or
- the Group Object service has received from the application layer
    3. a request to read the Group Object’s value
    4. an update on the Group Object's value.

The interaction between the application and the Group Object server are equivalent to the service primitives for request and indication, say it that in this case the exchange is subject to the status of the communication flags. This enables to distinguish local access from network wide update.

It is the responsibility of the internal or external user application program to trigger Group Object value transmissions.

3.3.2 Reading the Group Object Value

The following diagram gives the process flow when an application reads a Group Object value:



a If a negative A_GroupValue_Read.Lcon is received then the communication status shall be set to "error"; the actions following will not happen (no new value, no "update" flag).

b Update is subject to the “Update Enable” and “Communication” flag when used, else to the “Write Enable” flag.

Figure 3 - Reading a Group Object Value

3.3.3 Receiving a Request to Read the Group Object Value

The following diagram sketches the process flow when the Group Object Server receives a request to read a Group Object Value.

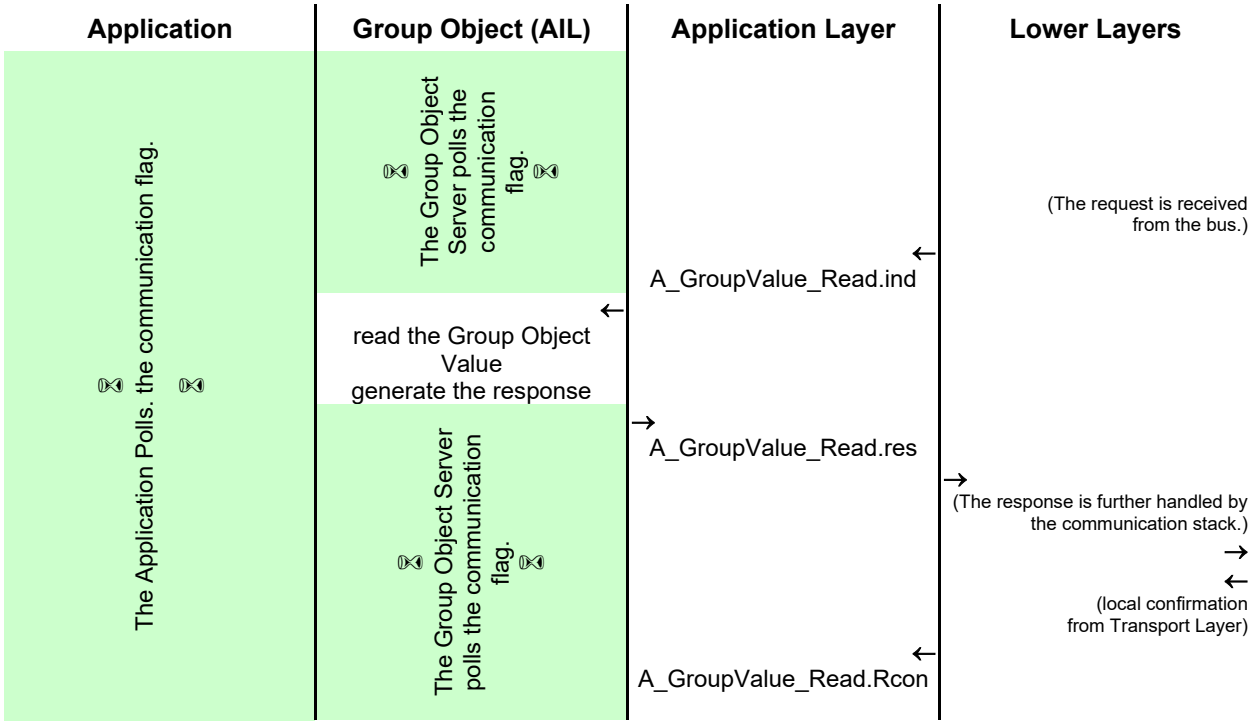
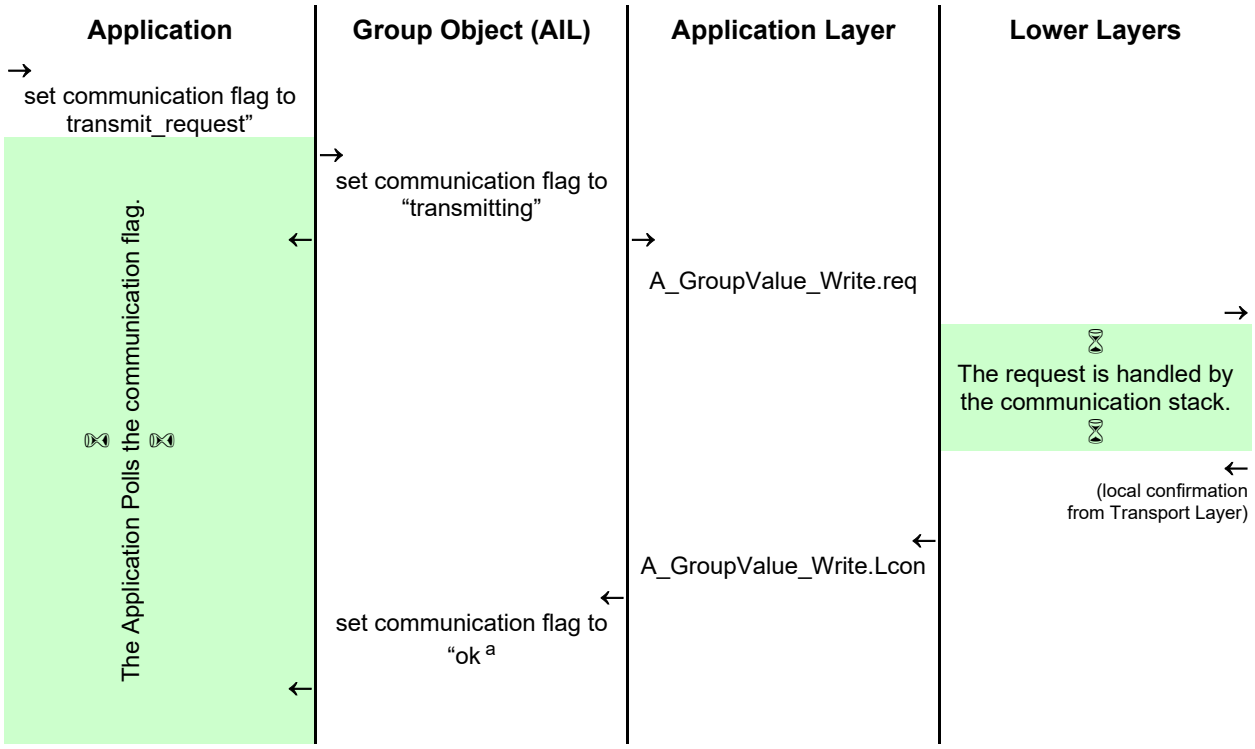


Figure 4 - Receiving a Request to read the Group Object Value

3.3.4 Writing the Group Object Value

The following diagram gives the process flow when an application writes a new value to a Group Object.

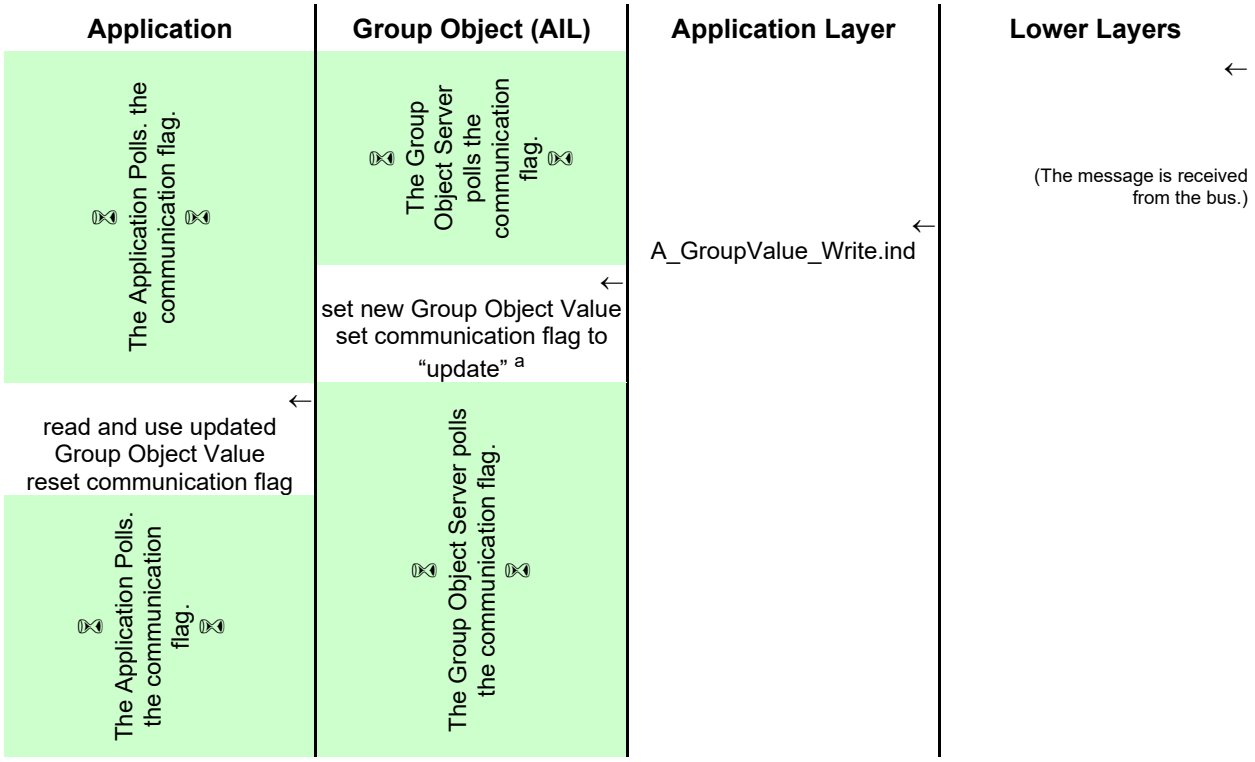


a If a negative A_GroupValue_Write.Lcon is received then the communication status shall be set to "error".

Figure 5 - Writing a Group Object Value

3.3.5 Receiving an Update on the Group Object Value

The following diagram sketches the process flow when the Group Object service receives a request to write a new value to the Group Object.



a This handling of the communication flag and the updating of the Group Object Value shall be subject to the configuration flag "write enable".

Figure 6 - Receiving an update of the Group Object Value

3.4 Group Object indirection – Group Object Handles and PID_OBJECT_VALUE (PID = 62)

Group Object Indirection denotes the requirement on the Application Interface Layer for certain Device Profiles to – next to the above specified mechanisms - provide an additional, parallel access to the values of the Group Objects in point-to-point connectionless or connection-oriented communication mode using the Function Property PID_OBJECT_VALUE,

In this, a Group Object shall be addressed in the device by a two octet Group Object Handle.

Please refer to the specification of PID_OBJECT_VALUE in [04].

4 Interface Object Server

4.1 Common structure

Interface Objects shall be instances of a common general structure. Each Interface Object Instance in a device shall have a unique identifier in the device, the Object Index.

Each Interface Object in a device shall be addressed by an Object Index or by an Interface Object Type and an Interface Object Instance. The Object Index shall be unique within the device. Interface objects of the same Interface Object Type shall be numbered by the Interface Object Instance starting with 1. So the combination of Interface Object Type and Interface Object Instance shall be unique. Each Property of an Interface Object shall be addressed with a Property Identifier. The Property Identifier shall be unique for the Interface Object.

Properties with a Property Identifier ≤ 255 shall have a lower Property Index than all properties with a Property Index > 255 as the Properties with a Property Identifier > 255 are only accessible with extended Property Services.

For the services A_PropertyDescription_Read and A_PropertyExtDescription_Read, a Property may be addressed also by the Property Index.

Each Interface Object shall at least contain the Property Object Type.

All Interface Object shall base on the common structure as presented in Figure 7. Depending on the flavour of the Interface Object, part of this structure may not be present.

Device				
	Interface Object			
		Property		
			Property description	
				Property Identifier (2 octet, unsigned12) = PID_OBJECT_TYPE
				Property Datatype (1 octet, unsigned8)
				DPT (4 octet, unsigned16 unsigned16)
				max_nr_of_elem (unsigned 16)
				Access (1 octet, unsigned4unsigned4)
			Property value	
				Array(0)=current nr. of elements (unsigned 16)
				Array(1 … max_nr_of_elem) = value
		…		
		Property		
			Property description	
				Property Identifier (2 octet, unsigned12)
				Property Datatype (1 octet, unsigned8)
				DPT (4 octet, unsigned16 unsigned16)
				max_nr_of_elem (unsigned 16)
				Access (1 octet, unsigned4unsigned4)
			Property value	
				Array(0)=current nr. of elements (unsigned 16)
				Array(1 … max_nr_of_elem) = value
		…		
		Property		
			Property description	
				Property Identifier (2 octet, unsigned12)
				Property Datatype (1 octet, unsigned8)
				DPT (4 octet, unsigned16 unsigned16)
				max_nr_of_elem (unsigned 16)
				Access (1 octet, unsigned4unsigned4)
			Property value	
				Array(0)=current nr. of elements (unsigned 16)
				Array(1 … max_nr_of_elem) = value

Figure 7 - Interface Object structure

From this common structure of Interface Objects, three types are derived.

1. Full Interface Objects, and
2. Reduced Interface Objects, and .
3. Extended Interface Objects.

These types are specified in the following clauses.

NOTE 1 If in the following specification the specific type is not explicitly mentioned or clear from the scope, then the requirements apply for both Full Interface Objects as well as for Reduced Interface Objects.

The Extended Interface Objects are an extension of Full Interface Objects. If a single device supports extended Interface Objects then it shall support this for all Interface Objects. There shall be no co-existence of other types.

If no Extended Interface Objects are supported then in a single device both other types of Interface Objects (Full - and Reduced) may co-exist.

This means that the following cases are possible.

1. A device contains only Reduced Interface Objects.
2. A device contains both Reduced - and Full Interface Objects. This means that the service A_PropertyDescription_Read shall be implemented in this device, but is only available for the Full Interface Objects.
3. A device contains only Full Interface Objects.
4. A device contains only Extended Interface Objects but all Access method as defined in Full Interface Objects also possible.

In [10] it is documented which combinations of Full - and Reduced Interface Objects are allowed.

4.2 Minimal requirements for Interface Objects

If any Interface Object is implemented in a device, then the Device Object shall be implemented. This Device Object shall have object_index 0.

In any Interface Object, the Property PID_OBJECT_TYPE is mandatory.

These rules apply both for mandatory Interface Objects as well as for optional Interface Objects (see below) and Extended Interface Objects.

4.3 Types of Interface Objects

4.3.1 Overview

Feature	Reduced Interface Objects	Full Interface Objects	Extended Interface Objects
Object			
Object Index	U_8	U_8	X
Property description			
Property Identifier	U_8	U_8	U_{12}
Property Index	X	U_8	U_{12}
Write enable flag	X	M	M
Property Datatype	X	M	M
Property Datapoint Type	X	O	M
read- and write access levels	X	M	M
Property Value			
max_nr_of_elem	X	U_{12}	U_{16}
current_nr_of_elem	U_{16}	U_{16}	U_{16}
nr_of_elem (in services)	U_4	U_4	U_8
start_index (in services)	U_{12}	U_{12}	U_{16}

Feature	Reduced Interface Objects	Full Interface Objects	Extended Interface Objects
Property services			
A_PropertyDescription_Read	X	M	M
A_PropertyValue_Read	M	M	M
A_PropertyValue_Write	M	M	M
A_FunctionPropertyCommand	X	O	O
A_FunctionPropertyState_Read	X	O	O
Extended Property Services			
A_PropertyExtDescription_Read	X	O	M
A_PropertyExtValue_Read	X	O	M
A_PropertyExtValue_WriteCon	X	O	M
A_PropertyExtValue_WriteUnCon	X	O	M
A_PropertyExtValue_InfoReport	X	O	O
A_FunctionPropertyExtCommand	X	O	O
A_FunctionPropertyExtState_Read	X	O	O

4.3.2 Full Interface Object

4.3.2.1 Definition

Full Interface Objects shall comply with the common data structure as specified in Figure 14 with the exception that the DPT is optional. Every Interface Object shall consist of a number of Properties. Every Property shall consist of

- one Property Description and
- one Property Value.

4.3.2.2 Property Description

The Property Description shall consist of

- the Property Identifier, and
- the Property Index, and
- the Write Enable flag, and
- the Property Datatype, and
- the Maximum Number of Elements information
- the definition of the read- and write access levels.

These elements are specified in the following paragraphs.

Property Identifier

The value of the Property Identifier shall be encoded in the field *property_id* of the A_Property-Description_Response-PDU.

Property Index

The Property Index of a Property shall be a unique number of for each within an Interface Object.

The first Property shall have Property Index 0 and further Properties shall be numbered with subsequent Property Indexes without gaps in the numbering.

The Property Index is used to read the Property Description in the A_PropertyDescription_Read-service.

Write Enable

The Write Enable flag shall specify whether the Property Value can be written through an A_PropertyValue_Write-service or not. The encoding shall be as follows:

0: The Property Value shall not be writeable, this is, the Property shall be read-only.

1: The Property Value shall be write-enabled.

Relation to access level: If this flag is cleared, the Property can not be written, independent of the set/used access level. If this flag is set, the access rights determine whether the Property can be written.

The value of the Write Enable flag shall be encoded in the field *w* of the A_PropertyDescription_-Response-PDU.

Property Datatype

The Property Datatype shall describe the data type of the Property. These Property Datatypes are specified in [09]. Every full Interface Object shall provide the most appropriate PDT for its Interface Object Properties, according the “Usage rules” specified there.

The value of the Property Datatype shall be encoded in the field *type* of the A_PropertyDescription_-Response-PDU.

Maximum Number of Elements

The value of a Property shall always be an array, as specified below. The Maximal Number of Elements shall specify the maximal number of elements of that array.

The value for max_no_of_elem shall be an unsigned 12 bit integer value with a 4 bit binary exponent without sign (the 4 most significant bits shall be the exponent).

The value of the Maximum Number of Elements shall be encoded in the field *max_nr_of_elem* of the A_PropertyDescription_Response-PDU.

Read - and write access levels

The attribute Access in the Property description shall indicate the necessary access level to read or write to the Property Value.

To obtain access to a protected Property Value, the Management Client shall apply the Management Procedure DM_Authorize2_RCo. This Management Procedure shall mainly be executed by the S-Mode Management Client when accessing devices of the Profiles System 2 and BIM M112 when the ETS®-user has provided an access key. This Management Procedure shall however also be followed by other Management Clients that support authorization. It is necessary when factory-released products need in a subsequent DM_SetKey to be locked by a key.

Table 1 lists the recommended access levels in function of the audience (management client).

Table 1 – Use of access levels for different purposes and Profiles

purpose	audience	access level	
		4 levels	16 levels
read access	runtime	level 3	level 15
end-user adjustable parameters	controller	level 3	level 3
configuration	ETS	level 2	level 2
product manufacturer	DevEdit/TransApp	level 1	level 1
system manufacturer	DevEdit/TransApp	level 0	level 0

4.3.2.3 Property Value

The value of a Property shall be an array with array index 1 to max_nr_of_elem. The array element ´0´ shall contain the current number (unsigned16 if max_no_of_elem exponent is 0 else unsigned 32) of valid array elements. The array can be reset to no elements by writing zero on element ´0´. The array shall automatically be extended if an element is written beyond the currently last element, but within the maximum allowed number of entries.

The Property with property_id = 1 and index 0 is named the Interface Object Type (PID_OBJECT_TYPE) and shall contain the description of the Interface Object itself. This Property is mandatory for every Interface Object.

Array of Interface Objects

If the maximum number of elements of the Property Interface Object Type is greater than 1 the whole Interface Object shall be an array of this Interface Object. Each Property of the array Interface Object must have at least the number of elements or a multiple of the number of elements of the Property Interface Object Type (PID_OBJECT_TYPE).

4.3.3 Extended Interface Object

4.3.3.1 Definition

Extended Interface Objects shall in full comply with the common data structure as specified in Figure 14. Every Interface Object shall consist of a number of Properties. Every Property shall consist of

- one Property Description and
- one Property Value.

4.3.3.2 Property Description

The Property Description shall consist of all elements as specified in Full Interface Objects. Additionally it shall contain of

- the Datapoint Type

Datapoint Type (DPT)

The Datapoint Type shall describe the Datapoint Type according to which the Property is encoded. These Datapoint Type are specified in [08]. Every Extended Interface Object shall provide the correct DPT for its Interface Object Properties.

The value of the Datapoint Type shall be encoded in the fields *DPT (main number)* and *DPT (subnumber)* of the A_PropertyExtDescription_Response-PDU.

4.3.3.3 Property Value

The value of a Property shall be an array with array index 1 to max_nr_of_elem. The array element ´0´ shall contain the current number (unsigned16) of valid array elements. The array can be reset to no elements by writing zero on element ´0´. The array shall automatically be extended if an element is written beyond the currently last element, but within the maximum allowed number of entries.

4.3.4 Reduced Interface Object

4.3.4.1 Structure

A Reduced Interface Object shall only support a subset of the common data structure Interface Objects, as depicted in Figure 8.

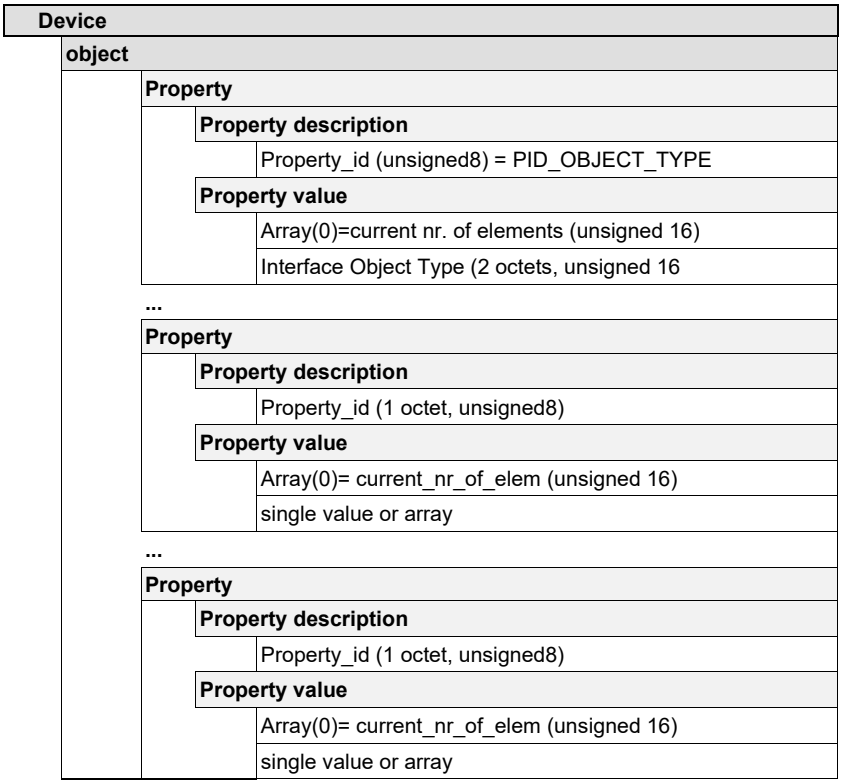


Figure 8 – Reduced Interface Object structure

This structure indicates that the Property Description shall be reduced to the Property_id. This Property_id shall not be readable by the A_PropertyDescription_Read-service, which is not supported for Reduced Interface Objects. The Property_id shall only be used for accessing the Property using the services A_PropertyValue_Read and A_PropertyValue_Write.

The support of the Reduced Interface Object requires the server side support of following APDUs:

A_PropertyValue_Read-PDU	(object_index [1 octet], property_identifier [1 octet], number_of_elements [4 bits], start_index [12 bits])
A_PropertyValue_Response-PDU	(object_index [1 octet], property_identifier [1 octet], number_of_elements [4 bits], start_index [12 bits] data [up to 10 octets])

A_PropertyValue_Write-PDU	(object_index [1 octet], property_identifier [1 octet], number_of_elements [4 bits], start_index [12 bits] data [up to 10 octets])

The behaviour of these Application Layer services is unchanged compared to the specification in [02].

The A_PropertyDescription_Read-service shall not be supported for a Reduced Interface Object. If a device receives an A_PropertyDescription_Read-PDU with the object_index referring to a Reduced Interface Object then the device shall show no reaction, this is, no A_PropertyValue_Response-PDU is generated.

4.3.4.2 Property Value is a single Value

The Property Value shall be accessed by the services A_PropertyValue_Read and A_Property-Value_Write. These services shall allow the handling of arrays, which means, that the PDUs shall always contain the fields nr_of_elem and start_index. In case of a single value, this shall be accessed by setting these fields to nr_of_elem = 1 and start_index = 1. A start_index = 0 shall refer to the current_nr_of_-elements, which shall be 1 in case of a single value. Therefore an A_PropertyValue_Read.req with nr_of_elem = 1, start_index = 0 shall result in an A_PropertyValue_Read.res(nr_of_elem = 1, start_index = 0, data = 1) in this case.

4.3.4.3 Property Value is an Array

In case the Property value contains an array, this shall have the following structure:

Array(0) = current_nr_of_elem (2 octets, unsigned 16)
Array(1 … max_nr_of_elem) = value

This means that element zero of the array shall always contain the current number_of_elem, stored in two octets (independent of the datatype of the array). Elements 1 and further shall contain the array elements.

NOTE The datatype and the max_nr_of_elements must be known by the client as this information cannot be retrieved for Reduced Interface Objects (the A_PropertyDescription_Read-service is not available).

4.3.4.4 Comparison to full Interface Objects

In comparison with Full Interface Objects the following information shall not be available due the lack of the A_PropertyDescription_Read-service:

- Property Datatype (PDT),
- access levels,
- maximum no. of elements in case of an array,
- access to Properties by the property_index

  In case of Full Interface Objects an A_PropertyDescription_Read can use the property_index to obtain the property_id. This is not possible for Reduced Interface Objects.

This means that a Management Client cannot retrieve this information from the device, but must have implicit knowledge about:

- which Properties exist in a Reduced Interface Object, and
- which is the Property Datatype (PDT) of each Property, and
- whether or not the Property contains a single value or an array, and
- the maximum no. of elements in case the Property value is an array what is, and
- what are the access levels.

  Properties of Reduced Interface Objects shall be accessible without prior authorisation. Reading and writing the Property value shall be possible with the lowest access level.

4.3.5 Error handling

For the services A_PropertyValue_Read and the A_PropertyValue_Write the same error handling shall apply as for full Interface Objects. This means that if an A_PropertyValue_Read-PDU or an A_PropertyValue_Write-PDU contains a field with an invalid value, the device shall answer with an A_PropertyValue_Response-PDU containing the same parameters, but an empty data field.

Please refer to the full error handling of these Application Layer services in Chapter 3/3/7 “Application Layer”.

This behaviour shall apply in the following cases.

- The Interface Object with requested object_index does not exist in the device [invalid object_index].
- In the requested Interface Object there is no Property with the requested Property_id [invalid property_id].
- The field nr_of_elem contains zero or a number that corresponds to an invalid nr_of_elements (i.e. >1 in the case of a single value or greater than the current_nr_of_elem above the start_index in case of an array) [invalid element count].
- The start_index is >1 in case of a single value or greater than the current_nr_of_elem in case of an array [invalid start_index].

If Extended Property Services are supported then Property arrays can be larger than 4 095 entries. If the description of such an Interface Object is requested by using the service A_PropertyDescription_Read then the answer shall contain as max_nr_of_elem 4 095.

4.4 Types of Properties

4.4.1 Data Properties

The Properties and the Property Description of Data Properties are typically accessed via the Application Layer services for Properties on a point-to-point connectionless - or connection oriented communication mode.

♦ **Services**

- A_PropertyValue_Read
- A_PropertyValue_Write
- A_PropertyDescription_Read

4.4.2 Function Properties

A Function Property shall allow a communication partner to call a function in the device. Function Properties shall be typed by the Property Datatype PDT_Function (value = 3Eh)..(Property Datatypes are specified in [09].)

♦ **Services**

The support of Function Properties requires the server side support of the following services.

- A_FunctionPropertyCommand-service

  This requires the support of the following APDUs:
  - A_FunctionPropertyCommand-APDU (object_index, property_id, data)
  - A_FunctionPropertyState_Response-APDU(object_index, property_id, return_code, data)

- A_FunctionPropertyState_Read-service
  This requires the support of the following APDUs:
  - A_FunctionPropertyState_Read-APDU (object_index, property_id, data)
  - A_FunctionPropertyState_Response-APDU(object_index, property_id, return_code, data)

Function Property Services shall be implemented and used as an entire "set". This is, the services A_FunctionPropertyCommand (1 service primitive) and A_FunctionPropertyState_Read (2 primitives) shall always be implemented together.

A Function Property shall be a Property within the same object model as a Data Property. A Function Property can be implemented within Full Interface Objects or Reduced Interface Objects. For Full Interface Objects the specific A_PropertyDescription_Read-service is as specified in [02].

Data

A Function Property differs from the above specified Data Property by the fact that the data transmitted to the Property by the Management Client *may* differ in format from the data responded by the Management Server as a response.

The format of data in request and response service primitives depends on the specific Function Property and is specified in [04].

The ASDU of the specified Function Property Commands is always specified to use the harmonized format as shown in Figure 9. The field "Reserved" is a placeholder for a return Code in the A_FunctionPropertyState_Response-APDU.

NOTE 2 There exist Function Properties that have a deviating format.

octet 10	octet 11	octet 12 ... octet n
Reserved	ServiceID	ServiceInfo
	WriteServiceID	Function dependent
00h	See below.	Function dependent

Figure 9 – Basic format for the Unified Function Property command for a WriteServiceID

octet 10	octet 11	octet 12 ... octet n
Reserved	ServiceID	ServiceInfo
	ReadServiceID	See below
00h	See below.	See below.

Figure 10 – Basic format for the Unified Function Property State read for a ReadServiceID

Reaction time for Function Properties

A common maximum reaction time of 1 second shall be maintained for Function Properties.

General guideline

If the data in the request, response and indication service primitives is

- the same, then the services
  - A_PropertyValue_Read
  - A_PropertyValue_Write

  should be used;

- different, then the services
  - A_FunctionPropertyCommand
  - A_FunctionPropertyState_Read

  should be used.

In general, Function Properties shall not be used during runtime. Exceptions shall be included in the Function Property definition. PID_OBJECT_VALUE (PID = 62) as specified in [04] may be used during runtime.

4.4.2.1 Data Property services vs. Function Property services

Properties of Interface Objects can be accessed either via Data Property services or Function Property services. The Property Datatype determines which of these service families shall give access to the Property.

Table 2 – Service access to Data - and Function Properties

Property Datatype	Device supports Standard Property services		Device supports Standard and Extended property services	
	Data	Function	Data	Function
PDT_FUNCTION	X	M	X	M
PDT_CONTROL	M	O	M [a]	M
Other	M	X	M	X

[a] PDT_CONTROL Properties should best be accessed by Extended Function Property services instead of Extended Data Property services. Access via Data Property services shall still be provided for compatibility reasons.

LEGEND: The standard symbols as defined in [10] clause 1.4 apply.

NOTE 3 For Properties of type PDT_CONTROL, Data Property services shall be provided for compatibility reasons only. Note that the response does not indicate the new status of the control Property as is available in Standard Property services for PDT_CONTROL Properties, so the new status has to be read explicitly afterwards. The recommended way to write to PDT_CONTROL Properties is via Extended Function Property services.

4.4.3 Network Parameter Properties

♦ **Services**

- A_NetworkParameter_Read
- A_NetworkParameter_Write

4.5 Ways for addressing Interface Objects

There are 2 different ways to address Properties of an Interface Object in a device.

1. → IndividualAddress → Interface Object Index →Property ID or
2. → Group Address → Object Instance →Property ID

To access the Property Description there are also two different ways:

1. → IndividualAddress → Interface Object Index → Property ID or
2. → IndividualAddress → Interface Object Index → Property Index

4.6 Interface Object Interworking

Standards for Interface Objects have been set for indication of the Property Datapoint Types, Property Identifiers and the Object Types. Please refer to [08] and [11].

4.7 System Interface Objects

System Interface Objects are Interface Objects designed for network - and application management. No System Interface Objects are mandatory for a device. These are the 4 most important System Interface Objects:

1. the Device Object
2. the Address Table object
3. the Association Table object
4. the Application Program object.

The purpose of System Interface Objects is to enable uniform Network Management. This shall support the end-user during the configuration of end devices through predefined System Interface Objects that offer the necessary Properties. See [03] for more details.

4.8 Defining Application Interface Objects

4.8.1 General

Standard Application Interface Objects are defined in the various application descriptions in the Chapters of Volume 7 ([11]).

It is possible to create proprietary Application Interface Objects. These shall be created according to the structures and datatypes given in clause 4.1 “Common structure” of this document.

Application Interface Objects may also be accessed by the Group Object Server via a reference in the Group Object Table.

4.8.2 Interface Object Server for own Application Interface Objects

Application Interface Objects have the same structure as System Interface Objects and therefore they use the same Interface Object Server.

4.8.3 Interface Object Client for Accessing Remote Application Interface Objects

An Interface Object Client can be external or internal.

An internal Interface Object Client may be based on the internal message format of A_Property-Description_Read, A_PropertyValue_Read and A_PropertyValue_Write request and confirmation messages.

The interface of the Interface Object Client at the external user application depends on the EMI chosen by the LM_Switch message, see [06] clause "Layer Access Management".

4.8.4 Message flow for Property Services

4.8.4.1 Scope

In the following paragraphs the message flow for Property handling is described in a client/server model.

4.8.4.2 Property Value Read

Interface Object Client		Interface Object Server
A_PropertyValue_Read.req →		
A_PropertyValue_Read.Lcon ←		→ A_PropertyValue_Read.ind
		← A_PropertyValue_Read.res
A_PropertyValue_Read.Acon ←		→ A_PropertyValue_Read.Rcon

The Application Layer User in the client shall use the A_PropertyValue_Read.req service-primitive to read the value of a Property of an Interface Object in the Interface Object Server. The Interface Object Server shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer in the Interface Object Client. The Interface Object in the Interface Object Server shall be addressed with an object_index and the Property of the Interface Object shall be addressed with a property_id. The nr_of_elem and start_index shall indicate the number of array elements starting with the given start_index in the Property Value that the user wants to read. The service shall be confirmed by the Application Layer in the Interface Object Client with an A_PropertyValue_Read.Lcon.

The user of Application Layer in the Interface Object Server shall receive an A_PropertyValue_Read.ind and shall respond with an A_PropertyValue_Read.res. The Transport Layer in the Interface Object Server shall transmit the A_PropertyValue_Read.res with a TSDU = A_PropertyValue_Response-PDU. The A_PropertyValue_Read.res service primitive shall be locally confirmed in the Interface Object Server with an A_PropertyValue_Read.Rcon.

The Application Layer User in the Interface Object Client shall receive an A_PropertyValue_Read.Acon with the requested data.

If the Interface Object Server has a problem to respond to the A_PropertyValue_Read.ind, e.g. Interface Object or Property does not exists or the requester has not the required access rights the nr_of_elements in the A_PropertyValue_Response-PDU shall be set to zero and shall contain no data.

4.8.4.3 Property Value Write

Interface Object Client		Interface Object Server
A_PropertyValue_Write.req →		
A_PropertyValue_Write.Lcon ←		→ A_PropertyValue_Write.ind
		← A_PropertyValue_Read.res
A_PropertyValue_Write.Acon ←		→ A_PropertyValue_Write.Rcon

The Application Layer User in the Interface Object Client shall use the A_PropertyValue_Write.req service primitive to write the value of a Property of an Interface Object in the Interface Object Server. The Interface Object Server shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer in the Interface Object Client. The Interface Object in the Interface Object Server shall be addressed with an object_index and the Property of the Interface Object shall be addressed with a property_id. The nr_of_elem and start_index shall indicate the number of array elements starting with the given start_index in the Property Value that the Interface Object Client wants to write. The service shall be confirmed by the Application Layer in the Interface Object Client with an A_PropertyValue_Write.Lcon.

The Application Layer User in the Interface Object Server shall receive an A_PropertyValue_Write.ind and shall respond with an A_PropertyValue_Read.res. The Transport Layer in the Interface Object Server shall transmit the A_PropertyValue_Read.res with a TSDU = A_PropertyValue_Response-PDU. The A_PropertyValue_Read.res service primitive shall be locally confirmed in the Interface Object Server with an A_PropertyValue_Write.Rcon.

The Application Layer User in the Interface Object Client shall receive an A_PropertyValue_Write.Acon with the written data.

4.8.4.4 Property Description Read

Interface Object Client		Interface Object Server
A_PropertyDescription_Read.req →		
A_PropertyDescription_Read.Lcon ←		→ A_PropertyDescription_Read.ind
		← A_PropertyDescription_Read.res
A_PropertyDescription_Read.Acon ←		→ A_PropertyDescription_Read.Rcon

The Application Layer User in the Interface Object Client shall use the A_PropertyDescription_Read.req service to read the value of a Property of an Interface Object in the Interface Object Server. The Interface Object Server shall be addressed with a local ASAP that shall be mapped to an Individual Address by the Transport Layer in the Interface Object Client. The Interface Object in the Interface Object Server shall be addressed with an object_index and the Property of the Interface Object shall be addressed with a property_id. The service shall be confirmed by the Application Layer in the Interface Object Client with an A_PropertyDescription_Read.Lcon.

The Application Layer User in the Interface Object Server shall receive an A_PropertyDescription_-Read.ind and respond with an A_PropertyDescription_Read.res. The Transport Layer in the Interface Object Server shall transmit the A_PropertyDescription_Read.res with a TSDU = A_Property-Description_Response-PDU. The A_PropertyDescription_Read.res service primitive shall be confirmed locally in the Interface Object Server with an A_PropertyDescription_Read.Rcon.

The Application Layer User in the Interface Object Client shall receive an A_PropertyDescription_-Read.Acon with the requested data.

5 File Server

5.1 File Server model

The File Server shall be coupled with the internal file system of the device and it shall communicate with the Application Layer via Application Layer services. An application only communicates with the internal file system.

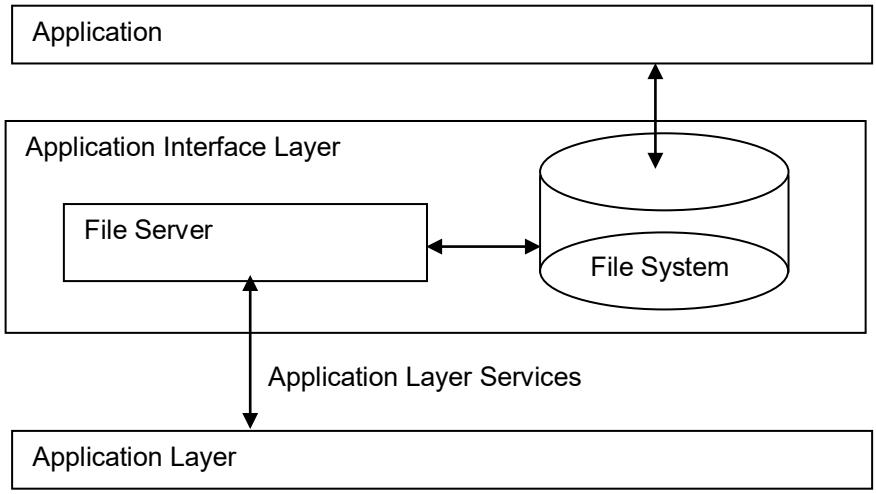


Figure 11 – Basic File Server model

In a closer look, the File Server itself consists of several independent blocks, which communicate with each other.

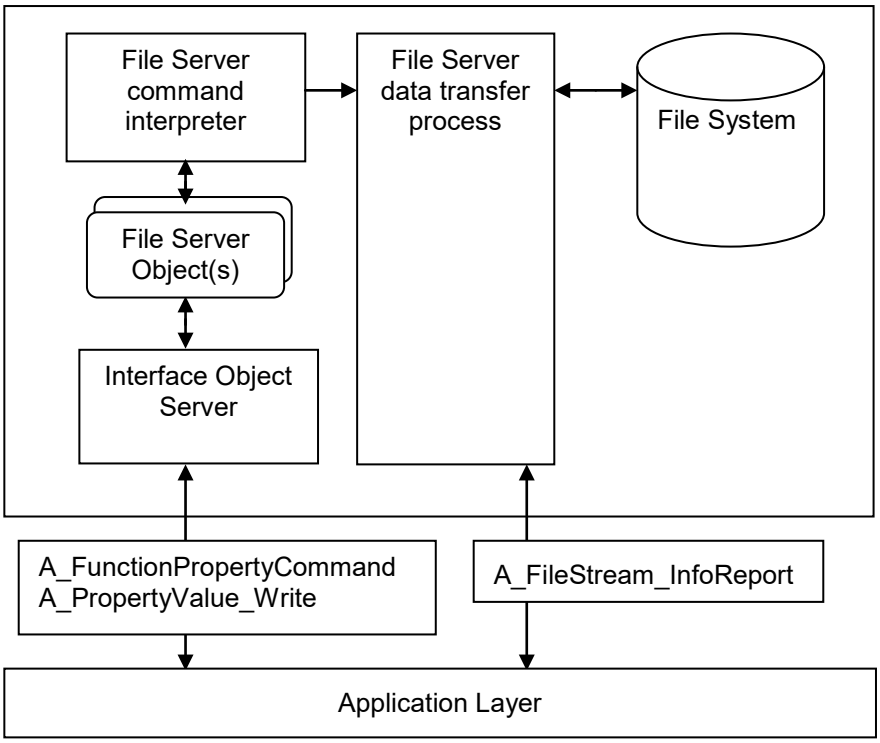


Figure 12 – Blocks of the File Server

The basic function of the File Server is to give a File Client access to the file system of the server. The client has to tell the server which file is of interest and what has to be done with it. For this communication, an Interface Object of type File Server Object shall be used. Properties of this Interface Object shall be used to structure this communication.

The Property PID_FILE_PATH shall contain the path name of the file to which the requested operations, e.g. file transfer request, apply.

Via the Property PID_FILE_COMMAND, a command for the file specified by the path name can be given. The command is always sent after the path name has been set.

It is also possible to include the path name into the command datagram. This is very useful, if long frames are used. For short frames the path name can only have a maximum length of 11 octets if it is transferred with the command. In a short frame environment, it is recommended, that the Property PID_FILE_PATH is always used for the transmission of a path name. If a path name is transferred with a command, it will override the path name stored in the Property PID_FILE_PATH.

To allow simultaneous access to several files from different clients (or from the same client) the concept of a File Handle shall be used. Before a client writes to the File Server Object, it retrieves a File Handle from this File Server Object. Once having a valid handle, the File Server Object is exclusively reserved for this client. All other clients will be blocked by getting an invalid file handle telling that this File Server Object is occupied.

When a File Server Object returns a valid File Handle to a File Client, it stores the Individual Address of this client. Further commands are only accepted from this client.

The reservation of a File Handle for a specific client shall be released either after the execution of a file command or after a time-out of approx. 6 s, if no command is executed i.e. the client does not communicate with this File Server Object.

A File Server Object shall only handle one file at a time. A File Server that can handle more than one File at a time shall implement as many instances of the File Server Object as files that it can handle simultaneously.

5.2 File Formats

5.2.1 General

File data shall always be stored and retrieved in binary form. During transmission no changes of file data are performed when using an FTP command.

By convention, text files use the PC standard CR/LF (0Dh, 0Ah) line termination.

5.2.2 Short HTML File Format

5.2.2.1 Format

Files containing HTML code are usually displayed in a browser window. It is convenient for the end user if the retrieval of HTML files is done in high speed. To achieve this, only a part of the whole file is transferred. If the “Get File” command is used, and the returned Content-Type is “text/short-html-xxx”, then the server expects the client to add a heading part and a footing part automatically.

```
<html>
        <head>
…
                <table align="center" border="0" width="640" bgcolor="#BFCFDF">
                        <tr>
```

Figure 13 – Heading Part for a short HTML File

```
                        <td align="center">
...
                        </td>
```

Figure 14 – Main Part of a short HTML File

```
                    </tr>
            </table>
…
            </body>
</html>
```

Figure 15 – Footing Part of a short HTML File

The transferred main part of the HTML file is only the contents of a predefined HTML table. The table definition is given in the heading part. The width of the table (640 in the example above) is defined by the returned Content Type (here: text/short-html-640).

5.2.2.2 Example

The example below shows a real HTML file with heading and footing part. It can be seen that the reduction in transfer is quite high.

```
<html><head>
<meta http-equiv="cache-control" content="no-cache">
</head>
<body bgcolor="#9AB3CD">
<center> </center>
<table align="center" border="0" width="690" bgcolor="#BFCFDF">
<tr><td align="center" valign="middle"><h2>
1.2.3
</h2></td></tr></table>
<center> </center>
<table align="center" border="0" width="640" bgcolor="#BFCFDF"><tr>
<!-- end of heading part -->

<td align="center">
<a href="cfga">[  A  ]</a> <a href="cfgb">[  B  ]</a>
<a href="cfgc">[  C  ]</a> <a href="cfgd">[  D  ]</a><br>
<a href="/1.2.3/en/cha">[home]</a>
<a href="/1.2.3/en/obj/obja">[objects]</a>
<p><b>  A  </b><p>
<form method="GET">
<input type="text" value="  A  " name="NA" size="11"><p>
<input type="submit" value="SET NAME (11)" name="NA">
</form><p>
(SET NAME stops application for 100ms)
<p></td>

<!—begin of footing part -->
</tr></table>
<center> </center>
```

```
<table align="center" border="0" width="690" height="30" cellpadding="3"
cellspacing="0"><tr>
<td align="center" bgcolor="#BFCFDF">
<h2><A HREF=".">zur&#252;ck</A></h2>
</td></tr></table>
<p align="center">&copy; 2006 Lingg &amp; Janke
</p></body></html>
```

5.2.3 Directory Listing Format

5.2.3.1 Introduction (informative)

Directory listings are dedicated files with a defined content. Two directory listing formats are defined.

1. The short format with a limited file size of 65 535 octets (16 bit length) and no date and time stamp. File names may have up to 255 characters.
2. The long format with a file size of up to 4 Gigabyte (32 bit length) and with date and time stamp. File names also may have up to 255 characters.

The format of the directory listing is the major limitation for the file system of the File Server. Limits are given for the maximum file size and the name length of file names and directory names.

For most small KNX devices it is sufficient to have a file size of 64 Kbyte. In a KNX TP1 bus environment, large files are very uncommon due to the long transfer times. And finally a clock is usually not provided to give files a date and time stamp.

For these devices, the short directory format is very suitable. Only the file size and the file name are transmitted. But, and this is the major advantage, directory entries with file names of up to 9 characters fit into one datagram of the A_FileStream_InfoReport service (if short frames are used). The transmission of a directory listing becomes very fast.

If the short directory format does not serve the needs of the KNX device, the long directory format shall be used.

The File Client shall be able to decode both directory formats. It is possible to distinguish both formats during the decoding process.

The transfer of a directory listing coded as described below is self optimizing and reduces busload to a minimum.

5.2.3.2 Short directory listing format

5.2.3.2.1 Transfer

Every file name or directory name in a short directory listing shall start with an A_FileStream_InfoReport datagram.

The whole directory listing shall be a sequence of A_FileStream_InfoReport datagrams.

The listing shall end if a A_FileStream_InfoReport datagram with data length 0 is sent.

5.2.3.2.2 Format

The first datagram shall contain an identifier, the file name length, the file size as an unsigned 16 bit integer and the file name or directory name. The identifier shall be used to distinguish whether the name is a file name or a directory name. And it also contains information about the type of the directory listing.

Limitations of the short directory listing format:

- max file size 64 Kbyte
- max name length for file names or directory names 255 characters
- no date and time

octet (in A_FileStream_InfoReport_PDU)	9	10	11	12	13 (MSB) to N
description	identifier	file name length	file size (binary)		file name / directory name (with short frames: first 9 characters)
		0 to 255	high	low	

Figure 16 – First Datagram of a short directory listing entry

octet (in A_FileStream_InfoReport_PDU)	9 (MSB) to 21
description	file name or directory name (with short frames: next 13 characters)

Figure 17 – Following Datagrams of a short directory listing entry (only for names longer than 10 characters if short frames are used)

♦ **Identifier**

The first octet shall be the identifier and shall be used for both directory listing formats.

bit	7 (msb)	6	5	4	3	2	1	0 (lsb)
description	0	0	0	0	0	0	LL	DN

Figure 18 – Identifier Octet

- DN (directory name)
  - 0 the following name shall be a file name
  - 1 the following name shall be a directory name
- LL (long listing)
  - 0 the following directory list shall be using the short directory listing format
  - 1 the following directory list shall be using the long directory listing format
- Unused bits shall be 0

Identifier	File name Length	Meaning
00h	09h	file name with 9 chars (fits into one datagram)
00h	23h	file name with 35 chars (two extra datagrams are needed if short frames are used)
01h	09h	dir name with 9 chars (fits into one datagram)
01h	16h	dir name with 22 chars (a second datagram is needed if short frames are used)

Figure 19 – Examples for the Identifier and File name Length octets in a Short Directory Listing

high	low	Meaning
00h	FFh	File size is 255 octets
04h	00h	File size is 1024 octets
80h	00h	File size is 32768 octets
FFh	FFh	File size is 65535 octets

Figure 20 – Examples for the unsigned 16 bit integer used for the file size

octet 13	octet 14	octet 15	octet 16	octet 17	octet 18	octet 19	octet 20	octet 21
61h	62h	63h	64h	65h	2Eh	74h	78h	74h
a	b	c	d	e	.	t	x	t

Figure 21 – Example of a file name that fits into the first short frame datagram. A leading "/" (slash) is not transmitted.

5.2.3.3 Long directory listing format

5.2.3.3.1 Transfer

Every file name or directory name in a long directory listing shall start with a new A_FileStream_InfoReport datagram.

The whole directory listing shall be a sequence of A_FileStream_InfoReport datagrams.

The listing shall end if a A_FileStream_InfoReport datagram with data length 0 is sent.

5.2.3.3.2 Format

The first datagram shall contain an identifier, the file name length, the file size encoded as a 32 bit unsigned integer, the file date (3 octets, DPT_Date, DPT_ID = 11.001), the file time (3 octets, DPT_TimeOfDay, DPT_ID = 10.001) and the filename or directory name. The identifier shall be used to distinguish whether the name is a filename or a directory name. It shall also contain information about the type of the directory listing.

Limitations of the long directory listing format:

- max file size 4 Gigabyte
- max name length for filenames or directory names 255 characters
- with date and time

octet (in A_FileStream_InfoReport_PDU)	9	10	11	12	13	14	15	16	17
description	identifier	file name length	file size (32bit unsigned integer)				Date (DPT_Date)		
		0 to 255	high	mhigh	mlow	low	Day	Month	Year

18	19	20	21 (MSB) to N
Time (DPT_TimeOfDay)			File name / directory name
Hour	Minutes	Seconds	

Figure 22 – First Datagram of a Long Directory Listing Entry

octet (in A_FileStream_InfoReport_PDU)	9 MSB to 21
description	file name or directory name (next 13 characters)

Figure 23 – Following datagrams of a long directory listing entry (only for names longer than 1 character if short frames are used)

The first octet shall be the identifier and shall be used for both directory listing formats.

bit	7 (msb)	6	5	4	3	2	1	0 (lsb)
description	0	0	0	0	0	0	LL	DN

Figure 24 – Identifier octet

- DN (directory name)
  - 0 the following name shall be a file name
  - 1 the following name shall be a directory name
- LL (long listing)
  - 0 the following directory list shall be using the short directory listing format
  - 1 the following directory list shall be using the long directory listing format
- Unused bits shall be 0

Identifier	File name Length	Meaning
02h	01h	file name with 1 character (fits into one datagram)
02h	1Bh	file name with 27 characters (two extra datagrams are needed if short frames are used)
03h	01h	directory name with 1 character (fits into one datagram)
03h	0Eh	directory name with 14 characters (a second datagram is needed if short frames are used)

Figure 25 – Examples for the Identifier and File name Length octets in a Long Directory Listing

high	mhigh	mlow	low	Meaning
00h	00h	00h	FFh	File size is 255 octets
00h	00h	FFh	FFh	File size is 65535 octets
80h	00h	00h	00h	File size is 2147483648 octets
FFh	FFh	FFh	FFh	File size is 4294967295 octets

Figure 26 – Examples for the unsigned 32 bit integer used for the file size

octet 21
61h
a

Figure 27 – Example of a file name that fits into the first short frame datagram. A leading "/" (slash) is not transmitted.

6 Data Security in the Application Interface Layer

6.1 General requirements and overview

The AIL shall have the following functionality.

- Support the Access Policies — see clause 6.2
- Support the Security Mode — see clause 6.2.2.1.3.1
- Handle Security Failures in the AIL — see clause 6.3

The following clauses specify these and other functions.

Note to the realisation

The below clauses give common requirements as a structured specification, differentiating between acceptance for services and Datapoints. However, within the S-AL, there are no interfaces (data interfaces, message interfaces, API etc.) and no Resources standardised. The below clauses thus give requirements to a black box behaviour.

6.2 Access Policies

6.2.1 Quick introduction (informative)

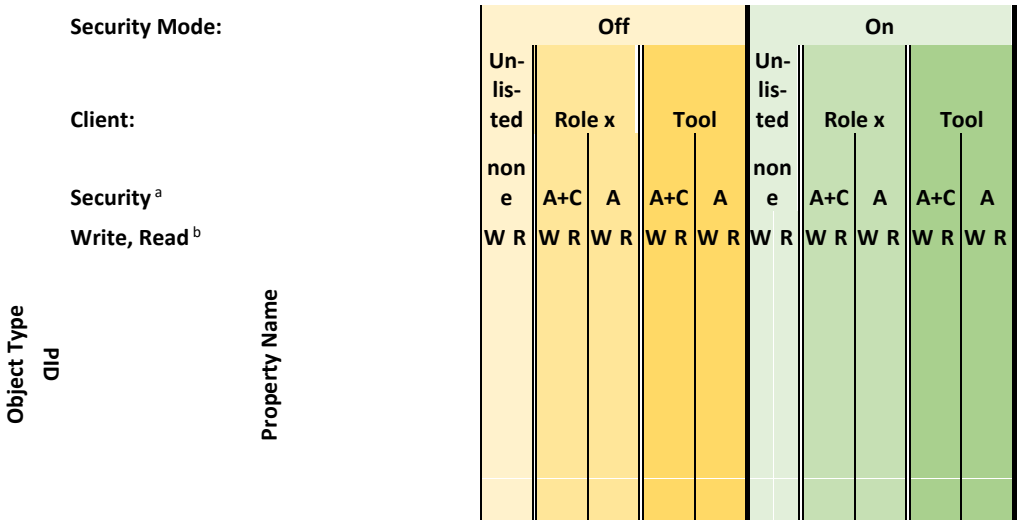
An Access Policy shall be a common definition of which communication partner shall have which read- or write permission to one or more services or Datapoints.

The notation of the Access Policies in this paper focusses on the minimal permissions for the KNX standard system Resources and is defined as follows.

Table 3 – Access Policies notation style (EXAMPLE)

Object Type	PID	Property Name	Security Mode:	Off								On									
			Client:	Un-listed		Role x				Tool				Un-listed		Role x				Tool			
			Security [a]	none		A+C		A		A+C		A		none		A+C		A		A+C		A	
			Write, Read [b]	W	R	W	R	W	R	W	R	W	R	W	R	W	R	W	R	W	R	W	R
8	68	PID_MAX_INTERFACE_APDU_LENGTH		1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1
8	69	PID_MAX_LOCAL_APDU_LENGTH		1	1	1	1	1	1	1	1	1	1	0	0	1	1	0	0	1	1	0	0
11	79	PID_TUNNELLING_ADDRESSES		0	1	0	1	0	1	1	1	1	1	0	0	0	1	0	0	1	1	0	0
11	91	PID_BACKBONE_KEY		0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0
11	92	PID_DEVICE_AUTHENTICATION_CODE		0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0
11	93	PID_PASSWORD_HASHES		0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0
11	94	PID_SECURED_SERVICE_FAMILIES		0	1	0	1	0	1	1	1	1	1	0	0	0	1	0	0	1	1	0	0
11	95	PID_MULTICAST_LATENCY_TOLERANCE		0	1	0	1	0	1	1	1	1	1	0	0	0	1	0	0	1	1	0	0
11	96	PID_SYNC_LATENCY_FRACTION		0	1	0	1	0	1	1	1	1	1	0	0	0	1	0	0	1	1	0	0
11	97	PID_TUNNELLING_USERS		0	1	0	1	0	1	1	1	1	1	0	0	0	1	0	0	1	1	0	0

3FF	/	1FF
3FF	/	0CC
15F	/	04C
008	/	008
008	/	008
008	/	008
15F	/	04C
15F	/	04C
15F	/	04C
15F	/	04C



Legend

a none, A, A+C

b: 0: not allowed, 1: alllowed

6.2.2 Permissions

6.2.2.1.1 Definition and overview

A Permission shall be the indication of whether no access, read access or write access shall be granted using a service or accessing data.

A Permission can be seen as a functional extension of the specification of the service of data.

Symbols

The Permissions are defined by the following symbols.

Table 4 – Permissions service definitions

Symbols	Intended service primitive
R: “Read”	• For a Datapoint, this shall mean that read access shall be allowed. • For a service, this shall mean that the following service primitives shall be accepted. This list is not complete. It is however mandatory for the listed services. - A_PropertyValue_Read.ind - A_FunctionPropertyState_Read.ind - A_PropertyExtValue_Read - A_FunctionPropertyExtState_Read - A_NetworkParameter_Read.ind - A_DomainAddressSerial_Number_Read.ind - and other.
W: “Write”	• For a Datapoint, this shall mean that write access shall be allowed. • For a service, this shall mean that the following service primitives shall be accepted. This list is not complete. It is however mandatory for the listed services. - A_PropertyValue_Write.ind - A_FunctionProperty_Command.ind - A_NetworkParameter_Write.ind - A_DomainAddressSerial_Number_Write.ind - A_PropertyExtValue_WriteCon.ind - A_PropertyExtValue_WriteUnCon.ind - A_FunctionPropertyExtCommand.ind - A_Restart.ind - and other.
R/W: “Read/Write”	• The Permissions for the following Property *Description* services shall be the logical OR operation of the Permissions for Read - and Write for that communication partner to the Property *Value*. This is, a request to any of these services by a communication partner shall be accepted if the communication partner has at least read - or write access to the Property Value. - A_PropertyDescription_Read - A_PropertyExtDescription_Read

NOTE 4 If there is no read – respectively write service primitive, then for this service primitive the Permission shall be noted as “not allowed” (0).

EXAMPLE 1 The A_Restart is considered as a write service primitive. There is no read service primitive concerning the restart. The indication “1” for the Read Permission for this service can thus not have any meaning and is don’t care.

The above classification of the Permissions focuses on the AL-services.

- This does not consider the S-A_Sync-service, which can by definition modify the own Sequence Number Sending of the requesting device (PID_SEQUENCE_NUMBER_SENDING) as well as the Last Valid SeqNr (PID_SECURITY_INDIVIDUAL_ADDRESS_TABLE) of the linked devices.
- This neither concerns the "cEMI services for local device management" (M_Prop_-) or the "Interface Feature services (USB, Tunnelling) - which are not passed over the Application Layer.

Notation style

A Permission shall be noted as a combination of firstly the Write allowance and then the read allowance. Table 5 lists the possible combinations.

Table 5 – Read – and Write Permissions

Symbol		Write access	Read access	Remarks
W	R			
0	0	Shall not be accepted	Shall not be accepted	The Role for which this is marked shall not have any access to this service or DP.
0	1	Shall not be accepted	Shall be accepted.	The Role for which this is marked shall have read access but shall not have write access.
1	0	Shall be accepted.	Shall not be accepted	The Role for which this is marked shall not have read access but shall have write access.
1	1	Shall be accepted.	Shall be accepted.	The Role for which this is marked shall have read access and write access.

6.2.2.1.2 Permissions for group communication

For group communication, the Group Key Table in the S-AL contains the GA_Index of the accepted GAs and secure Frames are additionally only accepted from senders of which the IA is in the Security Individual Address Table. It can thus be controlled which sender has access to which GO.

- However, this does not allow differentiating between the multicast services (A_GroupValue_Read and A_GroupValue_Write).

  EXAMPLE 2 It may be possible that a GO shall from any sender only be readable using A_GroupValue_Read, but that it shall only be writeable with A_GroupValue_Write using authentication. This cannot be controlled through the Group Key Table.

- This neither allows any further differentiation of any security conditions.

The S-AL controls the secure communication of a GA by a SA, but forwards the plain use of that same GA by a sender (IA) that is not in the Security Individual Address Table.

The Group Object Security flags allow setting the exact security requirements for Group Objects (authentication and confidentiality). The Group Object Security flags can be seen as a functional extension of the Group Object Config flags.

The permissions to GO access are limited to this.

If it is wanted that to a certain Datapoint differentiation be made in the multicast service, then this can be realised by implementing two different Datapoints.

EXAMPLE 3 If it is wanted that one group of senders to a Datapoint has write - and read access but another group only has read access, then this Datapoint can be implemented twice, as different Group Objects with the same data, but with different Group Object Config flags (read- and write- enable flags) and possibly different Group Object Security Flags.

6.2.2.1.3 Permissions for other communication modes

NOTE 5 This thus concerns the communication modes
point-to-domain, connectionless (broadcast),
point-to-all-points, connectionless (system broadcast),
point-to-point, connectionless, and
point-to-point, connection-oriented.

For the other communication modes, the Permissions can and shall be defined explicitly at the level of the service or at the level of the addressed Datapoint.

6.2.2.1.3.1 Security Mode

Definition

The Permissions for some services and data may depend on whether these services or data are accessed using plain communication or using KNX Data Security.

- Some data and services can and shall always be accessible using plain communication.
- Some data and services shall only be accessible using KNX Data Security.
- For some data and services, the Permissions depend on whether the Data Security in the device is effectively used and if secure links are established with the device or not. This dependency is defined as the *Security Mode*, which may have one of the following states and in function of which the Permissions are defined.

Security Mode
Disabled
Enabled

NOTE 6 For S-Mode implementations, the Security Mode shall be controlled through PID_SECURITY_MODE (see PID_SECURITY_MODE in [04]).

6.2.2.1.3.2 Permissions at service level

Permissions can be defined at the service level. This is possible for services that do not carry data or in case the permissions are independent of the data carried by the service.

EXAMPLE 4 The A_IndividualAddress_Read-service does not carry data and its Permissions are specified at service level (see Table 12).

EXAMPLE 5 The Permissions for the A_IndividualAddress_Write-service do not depend on the contained IA; the Permissions are thus also here defined at service level (see Table 12).

6.2.2.1.3.3 Permissions at data level

The Permissions shall be given in the specification of the data contained in the service Resource or Datapoint, through the indication of the Access Policy (see further).

The Permissions to data can be seen as a functional extension of the specification of that data.

EXAMPLE 6 The Permissions to a Property can be seen as a functional extension of the Property definition.

6.2.2.1.4 Mandatory aspects of Permissions

Storage of Permissions

There are no standard Resources for the implementation of Permissions. The internal storage format is implementation specific. The implementation may optimise in terms of size and evaluation speed and its relationship to the Roles in the Point-to-point Key Table (see below).

Evaluation and functioning

The AIL shall evaluate the Permissions to check whether a certain link and a certain service or access to data shall be accepted or not. If this evaluation is negative, then the handling shall be as follows.

- For Access Policies defined at service level:

  The service shall be ignored and the message shall not be passed to the AL. There shall be no negative confirmation of the AL-service to the requester.

- For Access Policies defined at data level:

  The normal error handling for the service shall apply.

EXAMPLE 7 An A_PropertyValue_Write.req or an A_PropertyValue_Read.ind will be confirmed with an A_PropertyValue_Response-PDU with nr_of_elem = 0 and no data.

Permissions for Role R_0 to R_{15}

This specification focusses on the required Permissions for Application Layer services and data for the management of the device. This specification therefore mainly focusses on the differentiation of the Permissions between the Role "Unlisted", "Role R_0 to R_{15}" and the Tool; it does not differentiate between the Roles 0 to 15: these all have the same mandatory Permissions concerning the access to management data and – services.

However, in the application, it is the intention that differentiation can and should be made between the Role R_0 to R_{15} concerning the access to the application specific data.

6.2.3 "Plain" access

"Plain" shall denote the plain access to the service or data, that is, without using the S-A_Data-service.

"Plain" shall not be mistaken for the common permitted access for secure communication (using S-A-Data), specifically, the Permissions listed under "Plain" shall not be interpreted as the Permissions to be granted for a failed secure access.

EXAMPLE 8 If a communication partner fails to authenticate itself in an S-A_Data-service with authentication only, then, the service will not be handled, as specified, even if the data that it wants to access would be specified as 11b for "Plain".

6.2.4 "Tool" access

With "Tool access" it is meant the access with the Tool Key and the flag "T" in the SCF set.

6.2.5 Roles

6.2.5.1 Definition and overview

There can be many Links using Secure Communication and there can be many Secure Datapoints in a secure application. A large amount of Permissions would have to be set if Permissions would have to be defined for each combination of Link and Datapoint (DP) or service.

Figure 28 – Many Links with many different Permissions cause a lot of definition data

Instead, it is however assumed that the Permissions to a DP may be the same for multiple Links. It may thus be better to group Links with the same Permissions in all DPs together, and define the Permissions for a group of Links, instead of for each separate Link. Such group of Links that have the same Permissions in a device is called a *Role*.

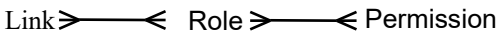


Figure 29 – The introduction of Roles reduces the amount of different Permission definitions for each DP

NOTE 7 The use of Roles has the following benefits.

- It keeps the size of the Permissions definitions smaller.
- It reduces the complexity. Instead of having an n-to-m relationship between Links and Permissions, it introduces an intermediate Level of 1-to-1 relationships.
- This eases the management and the administration.
- It is easier to understand for the ETS user.

6.2.5.2 Roles for group communication

The Permissions for a GO are fixed with the GO definition in the Group Object Security Flags independently of the communication partner. There is thus no need to defined Roles for group communication, as all communication partners all have the same Permissions.

Roles are thus not defined for group communication.

6.2.5.3 Roles for other system broadcast communication mode

The use of secure system broadcast communication mode is reserved for use with the Tool Key. The Permissions specified for the Tool Key in the Access Policies thus also considers the use of these communication modes.

6.2.5.4 Roles for other communication modes

NOTE 8 This thus concerns the communication modes
point-to-domain, connectionless (broadcast),
point-to-point, connectionless, and
point-to-point, connection-oriented.

For these communication modes, in the S-AL, the security shall amongst other be linked to the SA of the sender and the knowledge of the security key. This is supported through the following Resources.

- The *Security Individual Address Table*, and
- the *Point-to-point Key Table*

  The column *Roles* allows the AIL to differentiate in the authorisation between two or more authorised senders; without this, any sender that can properly secure its messages in the S-AL would further have full access to all the data and services in the AIL and Management.

EXAMPLE 9 The restart of a KNX S-Mode device is normally only executed by the MaC at the end of the configuration, or by the user, for diagnostic purposes. A restart excludes the MaS for some time from the runtime communication and may reset certain variables in the MaS. The restart may thus be a weakness in the security. Hence, it is better if the restart can only be requested by an authorised communication partner.

EXAMPLE 10 If an automation client authenticates itself to get access to the schedules of an HVAC control system, then - using this authentication - the automation client also has access to the linking and all other parameter values of that device. Vice versa, the properly authenticated ETS user will have access to the HVAC application parameters.

EXAMPLE 11 In a meter device, a MaC 1 may set tariff information and may read the registers for billing, but may not read the current consumption information. That is only available to a MaC 2, e.g. a display, which then again can only read, but not modify, the tariff information.

To be able to control which sender has what kind of access to which data and which services in the receiver, it is necessary that Permissions are defined.

6.2.5.5 Mandatory aspects of Roles

Evaluation of Roles in the KNX devices

Roles can be used in KNX Data Security but may also be used outside that scope.

In the receiver, each Link shall be associated to one or more Roles by setting the corresponding bits R_0 to R_{15} in the Point-to-point Key Table, which over the Permissions then finally controls the access to each separate DP or service.

The MaS shall evaluate the Roles as part of the handling of the incoming service primitives, this is, in the AIL or Management. These will only be verified if these messages are accepted and passed by the S-AL. This is, these flags will not be evaluated if there are errors in the SeqNr or the security (MAC verification, decryption).

The **sender** does not know what Role(s) it is assigned in the Receiver. It consequently does not know its Permissions in the receiver. This is a matter of configuration in the receiver. The sender cannot control this. Different senders can in the same device have the same or different Roles.

NOTE 9 The Role(s) of the sender is in the receiver linked solely to the sender's Individual Address. If in the sender there are multiple users that the sender wishes to differentiate in the receiver, then the sender has to manage these users and use a different source Individual Address for each of them.

Role "Tool"

Roles are not standardised within KNX. However, every KNX device shall at least foresee the Role "Tool".

EXAMPLES of Roles

- ETS - role
- Diagnostic Tool - role
- Visualisation Tool - role
- End User - role
- The heating technician - role
- The lighting controller - role
- A plain secure lighting sensor - role

Unlisted Role

Roles are related to links in the Security Individual Address Table, this is, can be defined for senders that at least authenticate themselves properly.

Messages may be received on Links that are not contained in the Point-to-point Key Table. In order to grant permissions to these senders, the implementation can use a role "Unlisted" (any sender that is not in the Security Pont-to-point Key Table). Obviously, these will be messages that are not protected using KNX Data Security. (S-AL will not forward these messages to P-AL). In order to grant or deny Permissions in such communication, this shall be assigned the Role "Unlisted".

Links in the Security Link Resource's using Plain communication

An implementation specific extension of the Point-to-point Key Table may consist of entries that do not use KNX Data Security. This is an extension of the above "Unlisted" Role and allows relating different, non-authenticated senders to different Roles.

Definition and assignment of Roles

The application developer defines the Roles: the number of Roles, their Permissions within the device and their dependency.

The MaC user assigns the Roles: which sender gets which Role.

For one Datapoint or service, there can be multiple Roles with different Permissions. However, different Roles may have the same Permissions in one DP or service; these Roles may then have different Permissions in other DPs or services.

Roles are not "access levels". There is no mandatory hierarchy between the Roles. As the Roles are chosen by the application developer, he can of course make them behave like access levels.

Roles – general requirements

NOTE 10 This clause gives the general requirements for the Point-to-point Key Table, as parameter within the AIL. For the specification as a standard Resource (Property PID_P2P_KEY_TABLE), please refer to Chapter 3/5/1 "Resources".

The Point-to-point Key Table shall define for each secure communication partner the Role(s) that shall be granted to it.

NOTE 11 This table may contain less entries than the Security Individual Address Table, which also contains the Source Addresses of communication partners that only perform group communication with the device.

The AIL shall evaluate the Roles in the Point-to-point Key Table only in reception direction. In transmission direction, any message shall just be sent out with its SA (IA) and DA (IA) as usual.

In case the Point-to-point Key Table foresees more than one Role for a Link Index, the Permissions Table may have to be evaluated for all Roles assigned to this Link Index. The AIL shall grant access to any DP or service if any assigned Role to the requesting IA is granted that access; the Permissions of an IA are thus the sum of the Permissions of all the Roles assigned to that IA.

EXAMPLE 12 In a heating system, the Role "*HVAC Technician*" may set the minimal – and the maximal water temperature in function of the dimensions of the HVAC installation and the building. The Role "*building owner*" cannot change these values, but may set the minimal – and maximal room temperature, to guarantee minimal comfort and prevent from excess energy consumption. The Role "renter" in turn can modify neither of these values, but may only access the small temperature shifts for comfort. In a larger, commercial building, these Roles will be executed by three different persons, each assigned his own Role. In a small building, that is configured and inhabited by the same person; this person will have the three Roles at the same time.

Two Links may be assigned the same Role.

Figure 28 shows an example of a Point-to-point Key Table.

IA_Index	Key (hex)	b_{15}	b_{14}	b_{13}	b_{12}	b_{11}	b_{10}	b_9	b_8	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
		R_{15}	R_{14}	R_{13}	R_{12}	R_{11}	R_{10}	R_9	R_8	R_7	R_6	R_5	R_4	R_3	R_2	R_1	R_0
2	5790DA550155FCD4501CEA888E8EA416	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1
3	AA4BE8C93CC0D9A4BA03BFECC61A38B9	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	1
5	D1315659C47907ACF04E1C7A6AC29BC2	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	1

Figure 30 – Example of Roles in the Point-to-point Key Table

NOTE 12

The Point-to-point Key Table is evaluated both by the S-AL for the keys as well as by the AIL for finding the Roles.

The S-AL forwards the Link Information as AL-service parameter, over the AL, to the AIL.

6.2.6 Access Policies

6.2.6.1 Notes

The Access Policies defined in this clause may be extended with new Access Policies as new use cases are solved with Data Security.

Additionally, the listed the Access Policies result from the security requirements of the standard KNX services and data and do not differentiate in the Permissions between the Roles R_0 to R_{15}, which may be needed for newer use cases and from application specific functionality.

For any Roles, a '1' in the write – or read column shall mean that it is allowed that any Role has write – respectively read – access; a '0' in a write - or read column shall however mean that none of the implemented Roles shall be granted the write – respectively read Permission.

Table 6 – Overview of the Access Policies (EXAMPLE)

Security Mode	Off										On										
Role	Un-listed		Role x				Tool				Un-listed		Role x				Tool				
Security	none		A+C		A		A+C		A		none		A+C		A		A+C		A		
Write and read	W	R	W	R	W	R	W	R	W	R	W	R	W	R	W	R	W	R	W	R	Hex notation style
	1	1	1	1	1	1	1	1	1	1	0	1	1	1	1	1	1	1	1	1	3FF / 1FF
	1	0	1	0	1	0	1	0	1	0	0	0	1	0	1	0	1	0	1	0	2AA / 0AA
	1	0	1	0	1	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	2AA / 008
	0	1	0	1	0	1	1	1	0	1	0	1	0	1	0	1	1	1	0	1	15D / 15D
	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	155 / 155
	0	1	0	1	0	1	0	1	0	1	0	0	0	0	0	0	0	1	0	0	155 / 004
	0	0	0	1	0	0	1	1	0	0	0	0	0	1	0	0	1	1	0	0	04C / 04C
	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	1	1	0	0	00C / 00C
	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	008 / 008
	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	004 / 004

6.2.6.2 Access Policies at service level

Please refer to Chapter 3/3/7 “Application Layer” ([02]) to the clause “Access Policies at service level”.

6.2.6.3 Access Policies at data level

6.2.6.3.1 Data accessed by the services A_DomainAddressSerialNumber_Read and A_DomainAddressSerialNumber_Write

Table 7 – Access Policies for Domain Address Serial Number services

Service primitive and accessed data	Access Policy
A_DomainAddressSerialNumber_Read - 2 octet DoA (PL110 DoA)	3FF / 3FF
A_DomainAddressSerialNumber_Write - 2 octet DoA: (PL110 DoA)	3FF / 00C
A_DomainAddressSerialNumber_Read - void: service not supported for 4 octet DoA (IP Domain Address: multicast address): service remains void.	- / -
A_DomainAddressSerialNumber_Write - 4 octet (IP Domain Address: multicast address)	3FF / 00C
A_DomainAddressSerialNumber_Read - if MaS has 6 octet DoA (KNX RF DoA)	3FF / 3FF
A_DomainAddressSerialNumber_Write - 6 octet (KNX RF DoA)	3FF / 00C
A_DomainAddressSerialNumber_Read - void: service not supported for 21 octet DoA (IP Domain Address: multicast address, Routing Security Version and Backbone Key)	- / -
A_DomainAddressSerialNumber_Write - 21 octet (IP Domain Address: multicast address, Routing Security Version and Backbone Key)	00C / 00C

6.2.6.3.2 Data Access by the Network Parameter services and the System Network Parameter services

This concerns the following services:

- A_SystemNetworkParameter_Read
- A_SystemNetworkParameter_Write
- A_NetworkParameter_Read
- A_NetworkParameter_Write

These services use of the identifiers Object Type and Property Identifer to identify the parameters used in these services. These are however only used as identifiers and shall not be seen as accesses to the Interface Objects and Properties of which the identifiers are used. Consequently, the Access Policies given for these Properties in this document do not apply for these services; the Access Policies for these services are under work in the KNX System Group and will be specified as soon as possible.

6.2.6.3.3 Data accessed by the A_Restart-service

Table 8 – Access Policies for A_Restart

Service primitive and accessed data	Access Policy
A_Restart - Restart_type = 0	3FF / 0CC
A_Restart - Restart_type = 1 - Erase Code 01h	3FF / 0CC
A_Restart - Restart_type = 1 - Erase Code 02h	3FF / 00C
A_Restart - Restart_type = 1 - Erase Code 03h	3FF / 000
A_Restart - Restart_type = 1 - Erase Code 05h	3FF / 00C
A_Restart - Restart_type = 1 - Erase Code 06h	3FF / 00C
A_Restart - Restart_type = 1 - Erase Code 07h	3FF / 00C
A_Restart - Restart_type = 1 - Erase Code 08h	3FF / 00C

6.2.6.3.4 Data accessed by A_PropertyDescription_Read and A_PropertyExtDescription_Read

The Access Policies for APCI_PropertyDescription_Read and APCI_PropertyExtDescription_Read are defined at data level and shall for each Property result from the OR function of the Access Policies to the write- and read access of the Property Value. This means that only if the communication partner would have the Permission to either read – or write the Property value, it will only have the Permission to read the Property description. If the communication partner does not have the Permission to either read – or write the Property Value, then it shall not have the Permission to read the Property description.

EXAMPLE 13 The Access Policies for PID_SEQUENCE_NUMBER_SENDING are 00C/00C. Regardless of the Security Mode (enabled or disabled), only the communication partner using the Tool Key can read the Property description, with A+C.

EXAMPLE 14 The Access Policies for PID_TOOL_KEY are 008/008. Regardless of the Security Mode (enabled or disabled), only the communication partner using the Tool Key can read the Property description, with A+C.

EXAMPLE 15 The Access Policies for PID_OBJECT_TYPE are 3FF/0CC. If Security Mode is disabled, any requester can read the Property description. If Security Mode is enabled, only the communication partner with the Tool Key are a Role Rx can read the Property description, only with A+C.

6.2.6.3.5 Data accessed by A_DeviceDescriptor_Read

Service primitive and accessed data	Access Policy
APCI_DeviceDescriptor_Read - any Device Descriptor Type	3FF / 0CC

6.2.6.3.6 Further Access Policies at Data Level to Configuration Parameters

Further Access Policies are defined in the specifications of the Resource in this paper in [06] and in general for all Resources in [12].

6.2.6.3.7 Access at Data Level to Application Parameters

With Application Parameters are meant the application specific Parameters as set by the MaC during the configuration.

- If Security Mode is disabled, then there are no access requirements for the Application Parameters.
- If Security Mode is enabled, then he Application Parameters shall be available to the Role “Tool” and may additionally be available to any secure Role R_0 to R_{15}. They shall not be available in plain.

6.2.6.4 Mandatory aspects of Access Policies

Number of Access Policies

There are no requirements concerning the number of Access Policies that are realised in a stack or device. This may at first result from the chosen Profile and from the security requirements of the application.

Resources for storage of Access Policies

There are no standard Resources for the implementation of Access Policies. The internal format is implementation specific. This can be hard coded and can be combined with the encoding of the Permissions.

The hexadecimal notation style and the representation style in Table 6 are no indications for any storage format. Additionally, these limit to the requirements on standard KNX services and – Resources and do not consider differentiation between the Permissions of Role R_0 to R_{15}! If an implementation supports R_0 to R_{15} then one or multiple ways to control efficiently the Access Policies will be needed.

The Access Policy definitions impose requirements on the implementation specific Permissions. The KNX application – and stack developer thus shall take care in the definition of the Permissions, that these Access Policy definition are respected.

6.2.7 Rules for Access Policies

These Rules do not express requirements to the implementation, but express general conditions for the current and future support of the Access Policies.

- If no Access Policy is defined for service or DP then the KNX stack – and application developer can set the Permissions for this service or DP implementation specific.

6.2.8 Management and Configuration

There are no standard Management – and Configuration requirements for the handling of the Access Policies.

Yet, it is allowed that the Access Policies data are modified as part of the Configuration Procedure. In S-Mode, this can be part of the memory image (memory mapped parameterisation) or realised through MergeIDs and Load Controls.

6.3 Register security failures in the AIL

The AIL shall register failures in the evaluation of Roles and Permissions in the appropriate counter in the SFL (see PID_SECURITY_FAILURES_LOG in [06]).